```
  priorityQueue ←
MinHeap()                                                    #2
  for (char, frequency) in charFrequenciesMap
do                      #3
    priorityQueue.insert(TreeNode([char],
frequency))                #4
  while priorityQueue.size > 1
do                                    #5
    left ←
priorityQueue.top()                                          #6
    right ←
priorityQueue.top()                                         #7
    parent ← TreeNode(left.chars + right.chars,
                      left.frequency +
right.frequency)              #8
    parent.left ←
left                                                         #9
    parent.right ←
right                                              #10

priorityQueue.insert(parent)
   #11
  return buildTable(priorityQueue.top(), [],
Map())                  #12
```

Each `TreeNode`, in fact, contains two fields (besides the pointers to its children): a set of characters, and the frequency of those characters in the text, computed as the sum of the frequencies of individual characters.

If you look at figure 2.15, you can see that the root of the final tree is the set of all characters in our example text, and hence the total frequency is 1.

This set is split into two groups, each of which is assigned to one of the root's tree, and so on each internal node is